INVESTIGANDO UM CLUSTER #1

Seu pod está em CrashLoopBackOff. Por onde começar?

O roteiro de 5 passos que uso em produção, sem chute.

```
$ kubectl get pods -n reports
NAME                  READY   STATUS             RESTARTS
pdf-exporter-v6grj    0/1     CrashLoopBackOff   23
```

Arraste para o lado →

CrashLoopBackOff não é o erro. É o sintoma.

O container inicia, cai, e o Kubernetes tenta de novo, esperando cada vez mais: 10s, 20s, 40s... até 5 minutos.



23 reinícios significa que o problema está acontecendo há horas, não agora.

> **O "BackOff" é só a espera entre as tentativas.** A pergunta certa não é "por que está em BackOff", e sim "por que o container sai".

1 Leia o motivo da última queda

O describe diz como o container morreu.

```
kubectl describe pod <pod> -n <ns>
# procure: Last State → Reason e Exit Code
```

1	Erro da própria aplicação: configuração, variável, dependência fora do ar
137	Morto pelo sistema (SIGKILL). Com Reason OOMKilled: estourou a memória
139	Falha de segmentação no binário
143	Recebeu SIGTERM: probe falhando ou desligamento

Role até o fim do describe. A seção Events mostra falha de probe, imagem que não baixa e o próprio BackOff.

2 Veja os logs de quem caiu

O log que importa é o do container anterior.

```
kubectl logs <pod> -n <ns> --previous
# -c <container> se o pod tiver mais de um
```

Sem o **--previous**, você lê o container novo, que acabou de subir e ainda não quebrou. É o erro mais comum de quem está começando.

> **Leia de baixo para cima.** A última linha antes da queda quase sempre diz o que faltou: uma variável, um arquivo, uma conexão.

3 Exit 137? Olhe a memória

Compare o uso real com o request e o limit.

POD	STATUS	REINÍCIOS	MEMÓRIA USO / REQ
coredns-tjnjt	Running	0	30 / 70 Mi
kafka-0	Running	0	10.579 / 12.288 Mi
pdf-exporter-v6grj	CrashLoop	**23**	**0 / 256 Mi**

Aqui o pod usa **0 Mi de 256 Mi**: ele nem chega a subir. Então não é falta de memória. A causa está nos logs e no que mudou.

> **OOMKilled de verdade** aparece com o uso encostado no limit e Reason OOMKilled no describe. O kafka-0 acima, perto do request, é quem merece atenção antes de cair.

4 Descubra o que mudou

Quase todo CrashLoop tem um "antes" e um "depois".

```
kubectl rollout history deploy/<app> -n <ns>
kubectl get events -n <ns> \
  --sort-by=.lastTimestamp
```

Imagem nova, ConfigMap, Secret, variável de ambiente, dependência atualizada. Ache o horário do primeiro reinício e compare com o último deploy.

> **Serviço fora do ar?** Primeiro devolva a versão anterior, depois investigue com calma. Estabilizar vem antes de entender.

5 Corrija, valide e registre

Resolver não é o fim. Registrar é.

```
kubectl rollout undo deploy/<app> -n <ns>
# ou corrija e faça o deploy de novo
```

Confira que os reinícios pararam e anote a causa e a prevenção:

EM ANDAMENTO

Corrigir CrashLoop do reports/pdf-exporter

Causa: a imagem nova quebrou a variável de ambiente do bucket. Ação: rollback, correção no pipeline e validação da variável antes do deploy.

Responsável: Bruno Costa · Prazo 26/10/2026

É esse registro que impede o mesmo incidente na próxima madrugada.

RESUMO PARA SALVAR

CrashLoopBackOff em 5 passos

1. describe: Reason e Exit Code
2. logs --previous
3. memória: uso × limit
4. o que mudou: rollout e events
5. corrigir, validar e registrar

Quer treinar isso com incidentes reais?

É assim que trabalho troubleshooting na Mentoria DevOps na Prática. Link no primeiro comentário.